gc_content.py 사용법 설명서

> lab11 · 생물정보학실험
> 대상 파일: `lab11/src/gc_content.py`

1. 프로그램의 목적과 기능

`gc_content.py`는 **FASTQ 파일의 read들을 GC 비율에 따라 두 그룹으로 나누고, 그룹마다 평균 품질 점수를 비교하는 프로그램**입니다.

DNA는 A, T, G, C 네 가지 염기로 이루어져 있습니다. 이 중 **G와 C가 차지하는 비율을 GC 비율(GC content)**이라고 합니다. G-C 쌍은 수소 결합이 3개(A-T 쌍은 2개)라서 더 단단하게 붙어 있고, 그래서 GC가 아주 많은 부분은 시퀀싱 과정에서 **읽기 어려워 품질이 떨어지는 경우**가 있습니다. 이 프로그램은 "우리 데이터에서도 정말 그런가?"를 숫자로 확인해 줍니다.

프로그램이 하는 일은 네 단계입니다.

1. FASTQ 파일을 읽어서 (이름, 서열, 품질) 묶음의 목록을 만든다. → `read_fastq()`
2. read마다 **GC 비율**을 계산한다. → `gc_content()`
3. read마다 **평균 품질 점수**를 계산한다. → `mean_quality()`
4. GC 비율이 **50% 이상이면 "높은 그룹", 미만이면 "낮은 그룹"**에 품질 점수를 넣고, 두 그룹의 평균과 차이를 출력한다.

함수	입력	출력	하는 일
`read_fastq(filename)`	파일 경로	`(이름, 서열, 품질)` 목록	FASTQ를 4줄씩 끊어 읽기
`gc_content(seq)`	서열 문자열	GC 비율(%)	G+C 개수 ÷ A+T+G+C 개수 × 100
`mean_quality(qual)`	품질 문자열	평균 품질 점수	문자 → Phred 점수로 바꿔 평균

2. 실행 명령어와 사용 방법

`lab11` 폴더에서 아래처럼 실행합니다.

```
cd lab11
python3 src/gc_content.py 〈FASTQ 파일 경로〉
```

예시:

```
python3 src/gc_content.py data/SRR1553607_1.fastq
python3 src/gc_content.py data/SRR1553607_2.fastq
python3 src/gc_content.py data/trimmed.fastq
```

결과를 파일로 남기고 싶으면 `>`로 저장합니다.

```
python3 src/gc_content.py data/SRR1553607_1.fastq > doc/gc_quality_1.txt
```

- 입력 파일은 **하나만** 받습니다. (`sys.argv[1]`)
- 출력은 6줄짜리 요약입니다. (전체 read 수, 그룹별 read 수, 그룹별 평균 품질, 차이)

> **주의**
>
> - 파일 이름을 빼먹으면 `IndexError: list index out of range`가 납니다.
> - FASTQ는 **4줄이 한 read**라는 규칙을 그대로 믿고 읽습니다. 파일 중간에 빈 줄이 있거나 줄 수가 4의 배수가 아니면 오류가 나거나 결과가 틀어집니다.
> - 파일 전체를 `readlines()`로 한꺼번에 메모리에 올리므로 아주 큰 파일에는 적합하지 않습니다. (실습 파일 약 56 MB는 몇 초 안에 끝납니다.)
> - 품질 줄이 비어 있는 read가 있으면 `mean_quality()`에서 0으로 나누게 되어 `ZeroDivisionError`가 납니다. (실습 파일에는 그런 read가 없습니다.)

---

3. GC 비율 계산 방법

```python
def gc_content(seq):
    gc = seq.count("G") + seq.count("C")
    atgc = sum(seq.count(base) for base in "ATGC")
    if atgc == 0:
        return 0
    return gc / atgc * 100
```

한 줄씩 보면 다음과 같습니다.

코드	뜻
`seq.count("G") + seq.count("C")`	서열 안의 G 개수 + C 개수
`sum(seq.count(base) for base in "ATGC")`	A, T, G, C 개수를 모두 더한 값
`if atgc == 0: return 0`	A/T/G/C가 하나도 없으면(예: 전부 N) 0으로 나누지 않도록 0을 돌려줌
`gc / atgc * 100`	비율을 **퍼센트(%)** 로 바꿈

```
GC 비율(%) = (G 개수 + C 개수) ÷ (A + T + G + C 개수) × 100
```

분모가 "서열 길이"가 아니라 "A+T+G+C 개수"라는 점이 중요합니다. 시퀀서가 어떤 염기인지 판단하지 못한 자리는 `N`으로 표시되는데, 이 `N`은 계산에서 빠집니다. 그래서 `N`이 섞여 있어도 GC 비율이 억울하게 낮아지지 않습니다.

또 `read_fastq()`에서 서열을 `.upper()`로 대문자로 바꿔 두었기 때문에, 소문자 `g`, `c`도 빠짐없이 세어집니다.

실제 예시 — `SRR1553607_1.fastq`의 첫 번째 read (길이 101)

```
GTTAGCGTTGTTGATCGCGACGCAACAACTGGTAAAGAATCTGGAAGAAGGATATCAGTTCAAACGCTCAAGCGAGATGATGGATATTTTTGAACGACTCA
```

- G = 26개, C = 17개 → G + C = 43개
- A + T + G + C = 101개 (N 없음)
- GC 비율 = 43 ÷ 101 × 100 ≈ **42.57 %** → 50 % 미만이므로 **낮은 그룹**

---

4. Phred+33을 이용한 평균 품질 점수 계산 방법

```python
def mean_quality(qual):
    scores = [ord(ch) - 33 for ch in qual]
    return sum(scores) / len(scores)
```

4-1. 품질 점수(Phred score)란?

FASTQ의 4번째 줄은 각 염기를 **얼마나 믿을 수 있는지**를 문자 하나로 적어 둔 것입니다. 이 숫자를 **Phred 품질 점수(Q)** 라고 하며, 오류 확률과 다음 관계가 있습니다.

```
Q = −10 × log10(오류 확률)
```

Q 점수	오류 확률	정확도
10	1/10	90 %
20	1/100	99 %
30	1/1,000	99.9 %
40	1/10,000	99.99 %

즉 **점수가 10 오를 때마다 오류가 10배 줄어듭니다.** 보통 **Q30 이상이면 매우 좋은 품질**로 봅니다.

4-2. Phred+33 변환

점수를 숫자 그대로 적으면 두 자리(예: `37`)가 되어 염기 1개와 짝이 맞지 않습니다. 그래서 **점수에 33을 더한 값의 ASCII 문자**로 바꿔 한 글자로 적습니다. 이것이 **Phred+33** 방식입니다. 거꾸로 문자에서 점수를 얻으려면 **ASCII 번호에서 33을 빼면** 됩니다.

- `ord(ch)` : 문자 `ch`의 ASCII 번호를 알려 주는 파이썬 함수
- `ord(ch) - 33` : Phred 점수

문자	ASCII 번호 ord()	Phred 점수 (−33)
`!`	33	0
`+`	43	10
`5`	53	20
`?`	63	30
`@`	64	31
`B`	66	33
`F`	70	37
`J`	74	41

4-3. read 하나의 평균 품질

`scores`는 read의 모든 위치 점수를 담은 리스트이고, `sum(scores) / len(scores)`로 평균을 냅니다.

실제 예시 — 위와 같은 첫 번째 read의 품질 줄

```
BB@FFFFFHHHHHJJJJJJJJJJJJJJJJJJJJGHIJJJJJJJJJJHHHHHFFFFFEEEEEEEEEDDDDDDDDDDDDDDDDDEDDDDEDEEEDDDDDDDDDD
```

- 앞부분: `B`=33, `B`=33, `@`=31, `F`=37, …
- 101개 점수의 합 = 3781
- 평균 = 3781 ÷ 101 ≈ **37.44** → 오류 확률 약 1/5,000 수준의 좋은 read

---

5. GC 비율 50% 이상과 미만으로 read를 구분하는 방법

```python
reads = read_fastq(sys.argv[1])

high_gc = []
low_gc = []

for name, seq, qual in reads:
    gc = gc_content(seq)
    q = mean_quality(qual)

    if gc >= 50:
        high_gc.append(q)
    else:
        low_gc.append(q)
```

1. 빈 리스트 두 개(`high_gc`, `low_gc`)를 준비합니다. 여기에는 **서열이 아니라 각 read의 평균 품질 점수(숫자)** 가 들어갑니다.
2. `for` 문으로 read를 하나씩 꺼내 GC 비율(`gc`)과 평균 품질(`q`)을 계산합니다.

3. `if gc >= 50:` 조건으로 나눕니다.
    - **50 이상** (50.0 포함) → `high_gc`에 `q` 추가
    - **50 미만** → `low_gc`에 `q` 추가

```
read 1:  GC 42.6%  Q 37.4  ──▶ low_gc  = [37.4]
read 2:  GC 51.5%  Q 34.0  ──▶ high_gc = [34.0]
read 3:  GC 63.4%  Q 30.1  ──▶ high_gc = [34.0, 30.1]
   ...
```

> **참고: 경계값 50%**
>
> 길이가 101(홀수)인 read는 GC가 정확히 50 %가 될 수 없습니다(50.5개가 될 수는 없으니까요). 그래서 원본 파일에서는 경계값에 걸리는 read가 0개입니다. 반면 `trimmed.fastq`는 길이가 50~101로 다양해서, **정확히 50 %인 read가 549개** 있고 이들은 `>=` 조건 때문에 **높은 그룹**에 들어갑니다.

---

6. 각 그룹의 평균 품질 점수 비교 방법

```python
print("전체 read 개수:", len(reads))
print("GC 비율 높은 read 수:", len(high_gc))
print("GC 비율 낮은 read 수:", len(low_gc))

if high_gc:
    print("GC 높은 그룹 평균 품질:", sum(high_gc) / len(high_gc))

if low_gc:
    print("GC 낮은 그룹 평균 품질:", sum(low_gc) / len(low_gc))

if high_gc and low_gc:
    diff = (sum(high_gc) / len(high_gc)
            - sum(low_gc) / len(low_gc))
    print("평균 품질 차이 (높음 - 낮음):", diff)
```

- `len(high_gc)`, `len(low_gc)` : 각 그룹에 들어간 **read 수**
- `sum(...) / len(...)` : 그룹 안 read들의 **평균 품질을 다시 평균**낸 값
- `if high_gc:` : 리스트가 비어 있으면 `False`가 되므로, **read가 없는 그룹은 건너뛰어** 0으로 나누는 오류를 막습니다.
- `diff = 높은 그룹 평균 − 낮은 그룹 평균`
    - **양수(+)** → GC 높은 read의 품질이 더 좋다
    - **음수(−)** → GC 높은 read의 품질이 더 나쁘다
    - **0에 가까움** → 차이가 거의 없다

> **"평균의 평균"이라는 점에 주의**

> 이 프로그램은 read마다 평균을 먼저 낸 뒤, 그 값들을 다시 평균 냅니다. 그래서 **긴 read든 짧은 read든 한 표씩** 똑같이 반영됩니다. 모든 염기를 한꺼번에 평균 내는 방식과는 값이 조금 다를 수 있습니다(길이가 다양한 `trimmed.fastq`에서 특히).

---

7. 실제 FASTQ 파일 실행 결과와 해석

7-1. 사용한 파일

파일	설명	read 수	read 길이
`data/SRR1553607_1.fastq`	원본 paired-end의 Read 1	203,445	101
`data/SRR1553607_2.fastq`	원본 paired-end의 Read 2	203,445	101
`data/trimmed.fastq`	Read 1을 fastp로 정리한 결과	188,357	50~101

7-2. 실행 결과

```
$ python3 src/gc_content.py data/SRR1553607_1.fastq
전체 read 개수: 203445
GC 비율 높은 read 수: 165738
GC 비율 낮은 read 수: 37707
GC 높은 그룹 평균 품질: 33.22470423018749
GC 낮은 그룹 평균 품질: 34.43877059358424
평균 품질 차이 (높음 - 낮음): -1.2140663633967463
```

```
$ python3 src/gc_content.py data/SRR1553607_2.fastq
전체 read 개수: 203445
GC 비율 높은 read 수: 166767
GC 비율 낮은 read 수: 36678
GC 높은 그룹 평균 품질: 30.30759237394535
GC 낮은 그룹 평균 품질: 33.20993970000635
평균 품질 차이 (높음 - 낮음): -2.902347326061001
```

```
$ python3 src/gc_content.py data/trimmed.fastq
전체 read 개수: 188357
GC 비율 높은 read 수: 154328
GC 비율 낮은 read 수: 34029
GC 높은 그룹 평균 품질: 34.71608536254968
GC 낮은 그룹 평균 품질: 35.99807539765036
평균 품질 차이 (높음 - 낮음): -1.2819900351006837
```

7-3. 한눈에 보는 요약 (소수점 둘째 자리 반올림)

파일	높은 그룹 read 수 (비율)	낮은 그룹 read 수 (비율)	높은 그룹 평균 Q	낮은 그룹 평균 Q	차이
SRR1553607_1	165,738 (81.5 %)	37,707 (18.5 %)	33.22	34.44	**−1.21**
SRR1553607_2	166,767 (82.0 %)	36,678 (18.0 %)	30.31	33.21	**−2.90**
trimmed	154,328 (81.9 %)	34,029 (18.1 %)	34.72	36.00	**−1.28**

7-4. 해석

① **이 데이터는 GC가 많은 편이다.** 세 파일 모두 read의 **약 82 %가 GC 50 % 이상** 그룹에 들어갑니다. 따로 계산해 보면 read들의 GC 비율 중앙값은 약 **60 %** 입니다. 즉 "50 %"라는 기준은 이 데이터의 한가운데가 아니라 조금 아래쪽이라서, 두 그룹의 크기가 4:1 정도로 크게 차이 납니다.

② **GC가 높은 read일수록 품질이 조금 낮다.** 세 파일 모두 차이가 **음수(−)** 입니다. 즉 GC 높은 그룹의 평균 품질이 낮은 그룹보다 낮습니다. 이것은 1절에서 말한 것처럼 GC가 많은 부분이 시퀀싱하기 까다롭다는 일반적인 경향과 맞습니다.

GC 구간을 10 %씩 더 잘게 나눠 보면 이 경향이 더 뚜렷합니다. (보조 계산 결과, `SRR1553607_2`)

GC 구간	read 수	평균 Q
30~40 %	9,609	34.83
50~60 %	58,960	31.86
70~80 %	25,704	26.65
80~90 %	2,012	16.14
90~100 %	156	7.28

GC가 80 %를 넘는 극단적인 read들은 품질이 크게 떨어집니다. 다만 이런 read는 수가 적어서, 그룹 전체 평균에서는 차이가 1~3점 정도로만 나타납니다.

③ **Read 2가 Read 1보다 품질이 낮고, GC에 따른 차이도 더 크다.** Read 2는 두 그룹 모두 평균 품질이 Read 1보다 낮고, 차이도 −2.90으로 Read 1(−1.21)의 두 배 이상입니다. Illumina paired-end에서 **두 번째 read는 시퀀싱 후반에 읽히기 때문에 보통 품질이 더 낮은데**, 그 영향이 GC 높은 read에서 더 크게 나타난 것으로 볼 수 있습니다.

④ **트리밍 후 품질은 좋아졌지만, GC에 따른 차이는 남아 있다.** `trimmed.fastq`는 원본 Read 1보다 두 그룹 모두 평균 품질이 약 1.5점 올랐습니다(33.22 → 34.72, 34.44 → 36.00). fastp가 품질 나쁜 끝부분과 read를 잘라냈기 때문입니다. 그러나 두 그룹의 차이(−1.28)는 원본(−1.21)과 거의 같습니다. 즉 **트리밍은 전체 품질을 올려 주지만, GC 함량에 따른 품질 차이 자체를 없애지는 못합니다.**

⑤ **그래도 두 그룹 모두 품질은 양호하다.** 가장 낮은 값(Read 2 높은 그룹, 30.31)도 **Q30 이상**입니다. 평균으로 보면 염기 1,000개 중 1개 정도 틀리는 수준이라, 이 데이터를 분석에 쓰는 데 큰 문제는 없습니다. 다만 GC가 아주 높은 read는 품질이 많이 낮으므로, 그런 read가 많은 분석에서는 따로 확인하는 것이 좋습니다.

> **한계**
>
> - 차이가 음수라는 것은 **경향**일 뿐, 통계적으로 의미 있는 차이인지는 이 프로그램만으로 알 수 없습니다(검정을 하지 않음).
> - 그룹을 50 % 하나로만 나누기 때문에 극단적인 GC 구간의 큰 품질 저하가 평균 속에 묻힙니다. 더 자세히 보려면 위처럼 구간을 여러 개로 나눠 보는 것이 좋습니다.

요약

항목	내용
실행	`python3 src/gc_content.py <FASTQ 파일>`
GC 비율	(G + C) ÷ (A + T + G + C) × 100, N은 제외
품질 점수	품질 문자의 `ord(문자) − 33` (Phred+33), read마다 평균
그룹 기준	GC ≥ 50 % → 높은 그룹, GC < 50 % → 낮은 그룹
비교	그룹별 평균 품질과 그 차이(높음 − 낮음) 출력
실습 결과	세 파일 모두 GC 높은 그룹 품질이 1.2~2.9점 낮음, Read 2에서 차이가 가장 큼